Machine Learning

Lecture 6: Machine learning for physics

Simon BERNARD
simon.bernard@univ-rouen.fr



AI for science

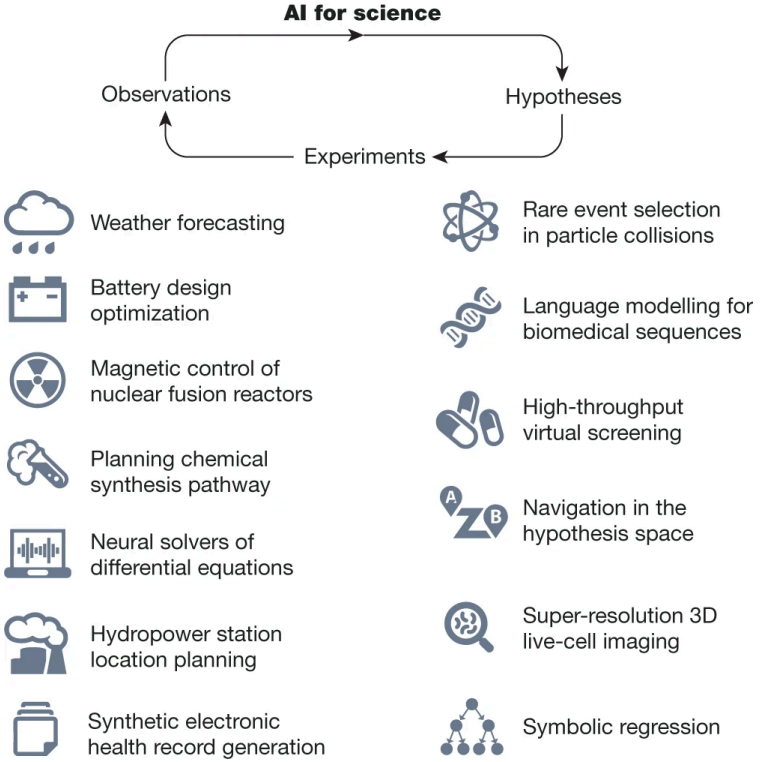
AI for Science

Major successes of AI: digital applications

- Images, videos, texts, video games, etc.
- Massively available data

What about sciences? Can we use AI:

- as an alternative to traditional approaches?
- to solve problems more effectively?
- to accelerate scientific discovery?

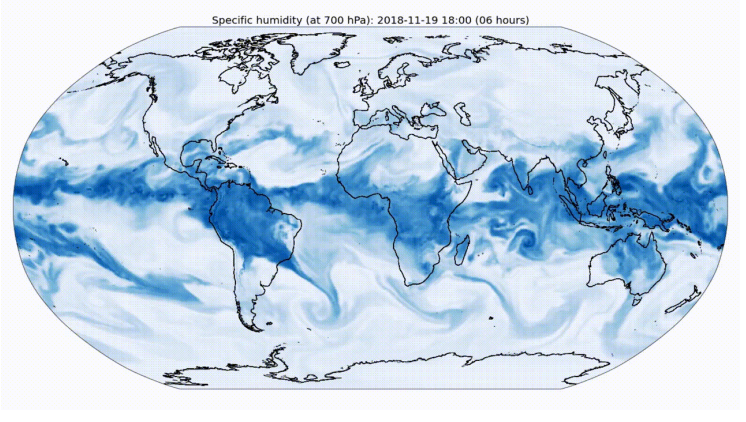


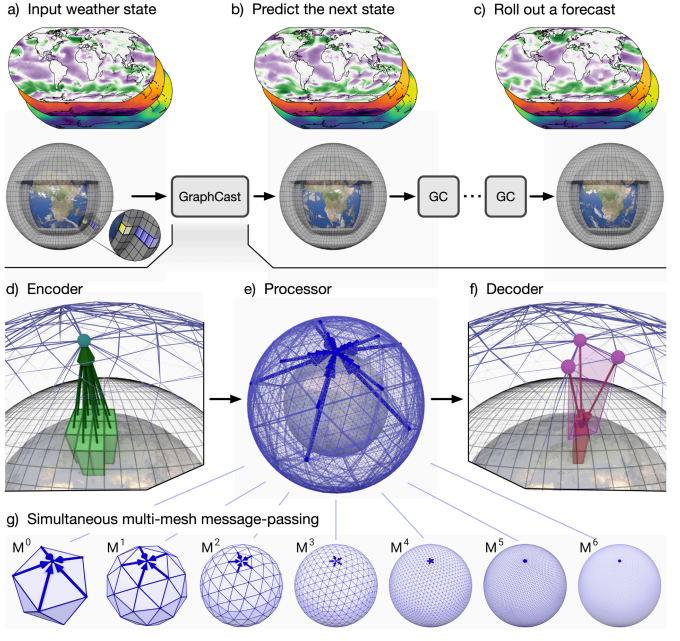
Wang et al, "Scientific discovery in the age of artificial intelligence", Nature, 2023

AI for Science: Notable recent works

Weather forecasting

- **GraphCast** (2023) [1]
- Training data: 40+ years of observations (ECMWF)
- Prediction of weather fields up to 10 days ahead
- 1.2 billion param., 128 TPUv4 chips for training





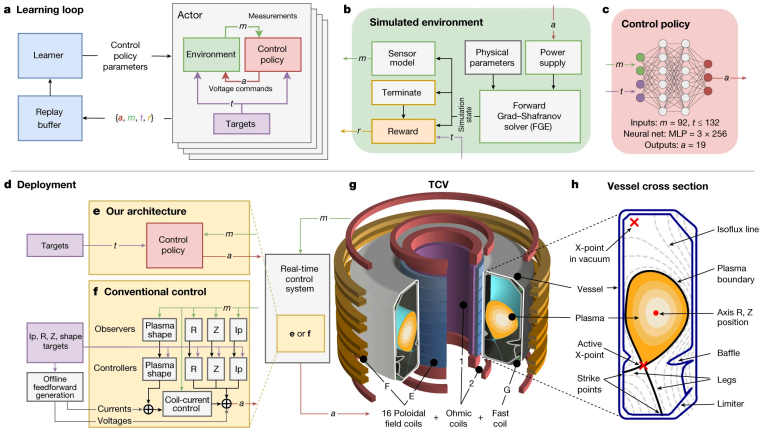
https://deepmind.google/blog/graphcast-ai-model-for-faster-and-more-accurate-global-weather-forecasting/

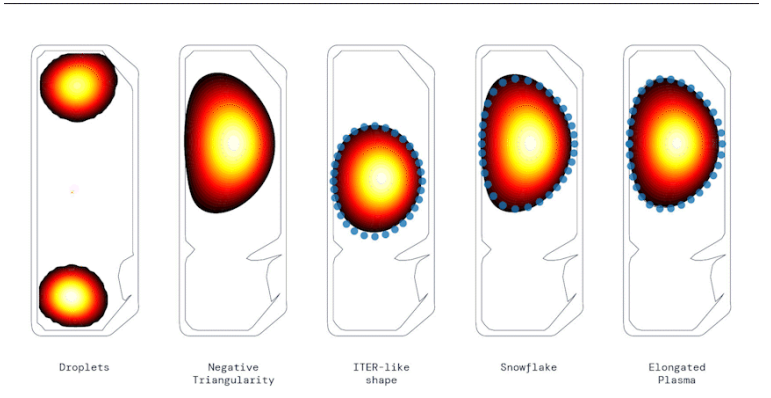
(1) Renamed with newer versions: GenCast in 2024 and WeatherNext in 2025 (github)

AI for Science: Notable recent works

Nuclear fusion for sustainable energy production

- High-performance, clean and virtually inexhaustible energy
- Very difficult to control due to the unstable state of the plasma used for fusion
- AI for modeling and controlling the plasma:
  - Based on reinforcement learning
  - Simulated data and control policy that require significant domain expertise



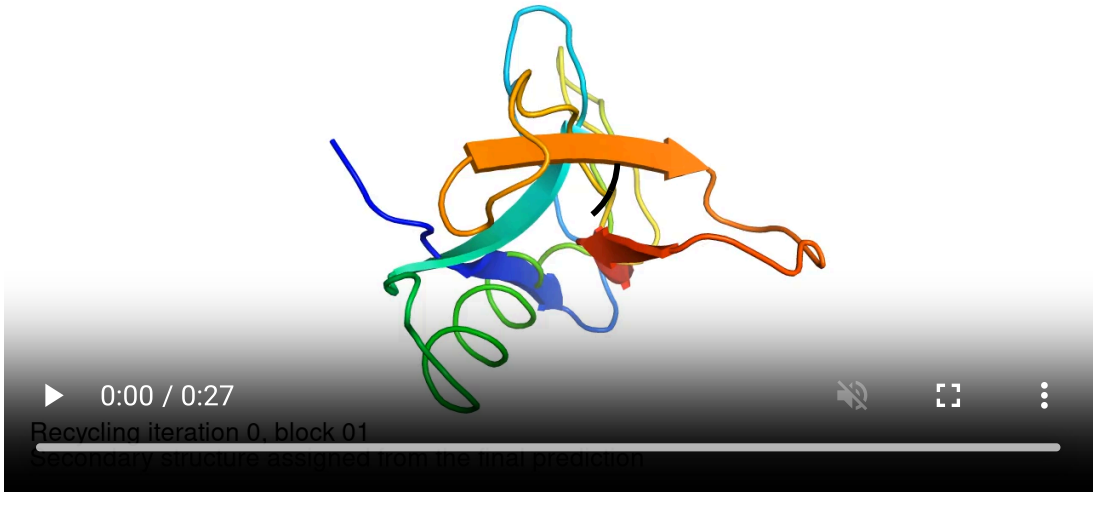


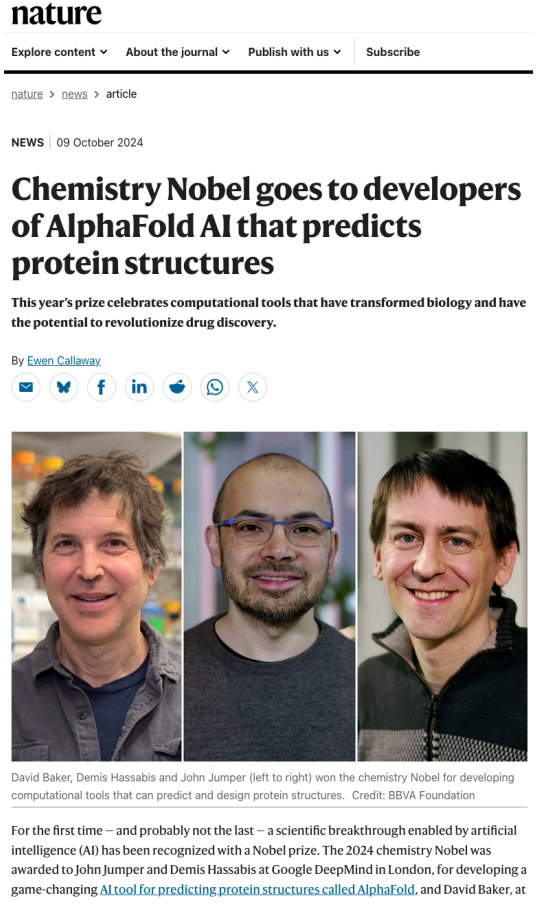
Degrave et al., "Magnetic control of tokamak plasmas through deep reinforcement learning", Nature, 2023

AI for Science: Notable recent works

Protein structure prediction

- **Alphafold 2 (2021) and Alphafold 3 (2023)**
- Trained on 200k+ known structures
- Predictions for almost all known proteins (~ 200M)
- Inspired by LLMs and with an iterative generation approach





nature

Explore content · About the journal · Publish with us · Subscribe

nature › news › article

NEWS | 09 October 2024

Chemistry Nobel goes to developers of AlphaFold AI that predicts protein structures

This year's prize celebrates computational tools that have transformed biology and have the potential to revolutionize drug discovery.

By Ewen Callaway

David Baker, Demis Hassabis and John Jumper (left to right) won the chemistry Nobel for developing computational tools that can predict and design protein structures. Credit: BBVA Foundation

For the first time – and probably not the last – a scientific breakthrough enabled by artificial intelligence (AI) has been recognized with a Nobel prize. The 2024 chemistry Nobel was awarded to John Jumper and Demis Hassabis at Google DeepMind in London, for developing a game-changing AI tool for predicting protein structures called AlphaFold, and David Baker, at

Scientific Machine Learning

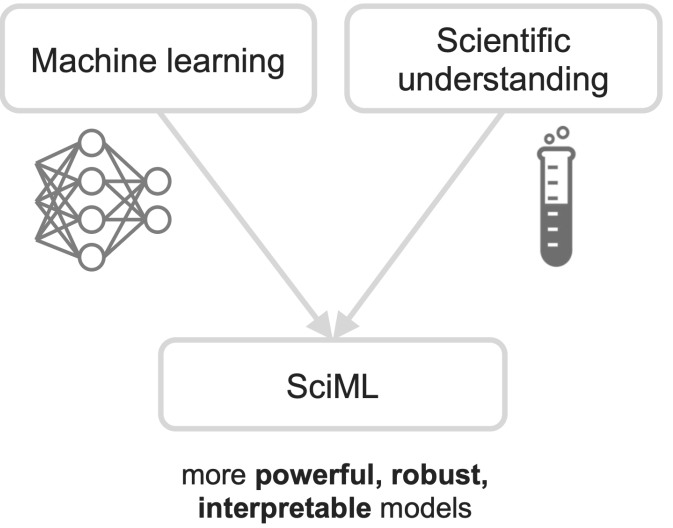
Limitations:

- Based on architectural priors
- Need for a large amount of data
- No interpretability and trustworthiness
- Poor generalization to new regimes

Contradictory to the scientific approach:

- Hypothesis, theory and experiments
- Must be interpretable and trustworthy
- Generalization to new regimes is the goal
- Must be consistent with scientific knowledge

Solution:



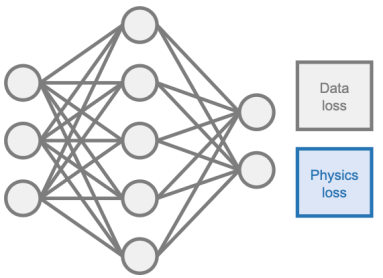
S. Mishra, B. Mosley, ETH Zürich, "AI in the Sciences and Engineering" (2024)

Scientific Machine Learning

Three main ways to incorporate scientific constraints

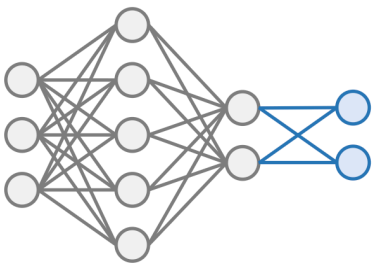
Soft constraint

Add a term to the loss function that penalizes violations of scientific laws (e.g. PDE residuals)



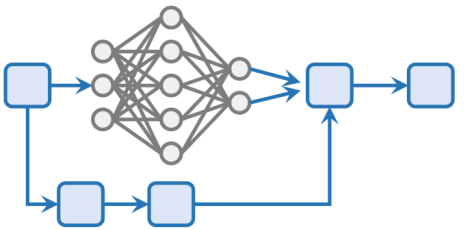
Hard constraint

Adapt the architecture (e.g. re-parametrizing the output) to ensure compliance with a scientific law



Hybrid

Integrate a model into a scientific process, such that ML is not a surrogate, but a building block



What are the machine learning tasks?

Partial Differential Equations are building blocks of science

- Partial derivative of $u(x_1, \ldots, x_n) : \mathbb{R}^n \to \mathbb{R}$:

$$\frac{\partial u}{\partial x_i} = \lim_{\Delta x_i \to 0} \frac{u(x_1, \ldots, x_i + \Delta x_i, \ldots, x_n) - u(x_1, \ldots, x_i, \ldots, x_n)}{\Delta x_i}$$

  i.e. the magnitude of variation of u along the x_i direction

- Partial differential equations (PDEs):

$$F\left(x_1, \ldots, x_n, u, \frac{\partial u}{\partial x_1}, \ldots, \frac{\partial u}{\partial x_n}, \frac{\partial^2 u}{\partial x_1^2}, \frac{\partial^2 u}{\partial x_1 \partial x_2}, \ldots\right) = 0$$

- Exemple: the heat equation

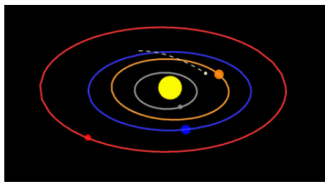
$$\frac{\partial u}{\partial t} - \alpha \nabla^2 u = 0, \quad u : \mathbb{R}^d \times [0, T] \to \mathbb{R}$$

  where u is the temperature field, α is the thermal diffusivity, and ∇^2 is the Laplacian operator

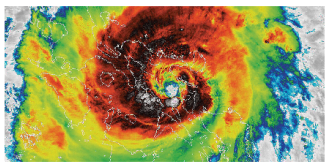
What are the machine learning tasks?

Partial Differential Equations are building blocks of science

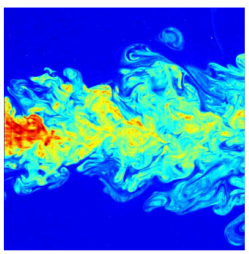
- Understanding and solving these equations is crucial for many scientific and industrial applications: aerodynamic simulation, weather forecasting, biological modeling, etc.
- **Numerical solvers are time-consuming, computationally intensive, and sometimes complex**



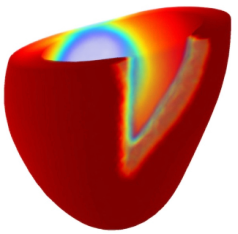
Solar system



Atmosphere



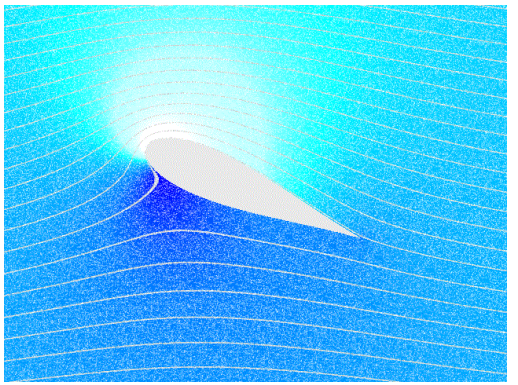
Fluid

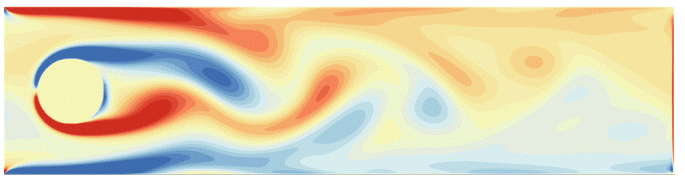


Heart electrical activities



Chemical reactions





Yuan YIN, soutenance de thèse, Sorbonne Université, 2023

What are the machine learning tasks?

Generic form of nonlinear PDEs:

$$\begin{cases} \mathcal{N}[u, \mathbf{x}, t; \mu] = 0, & \mathbf{x} \in \Omega \\ u(\mathbf{x}, t) = g(\mathbf{x}, t), & \mathbf{x} \in \partial\Omega \text{ (boundary condition)} \\ u(\mathbf{x}, 0) = u_0(\mathbf{x}), & \mathbf{x} \in \Omega \text{ (initial condition)} \end{cases}$$

where u is the unknown field, μ are the PDE parameters, $\mathcal{N}$ is a differential operator

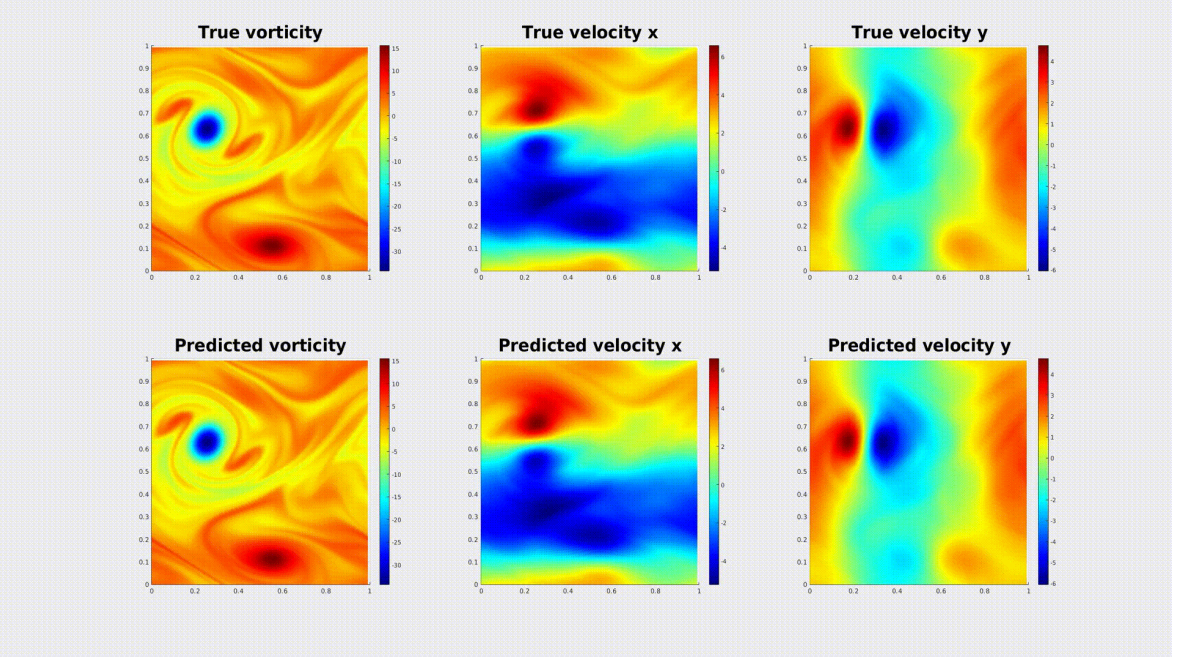
The three main tasks in scientific machine learning are:

- **Simulation**: solving a PDE for given initial and boundary conditions, i.e. finding u at any $(\mathbf{x}, t)$
- **Inverse problems**: inferring unknown parameters of a PDE from measurements, i.e. finding μ
- **Scientific discovery**: finding new scientific laws from data, i.e. finding $\mathcal{N}$

Example: Navier-Stokes equations for incompressible flow

$$\begin{cases} \dfrac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f} & \text{(momentum equation)} \\ \nabla \cdot \mathbf{u} = 0 & \text{(incompressibility condition)} \end{cases}$$

with $\mathbf{u}$ the velocity field, p the pressure, ν the kinematic viscosity and $\mathbf{f}$ the external forces



https://zongyi-li.github.io/neural-operator/

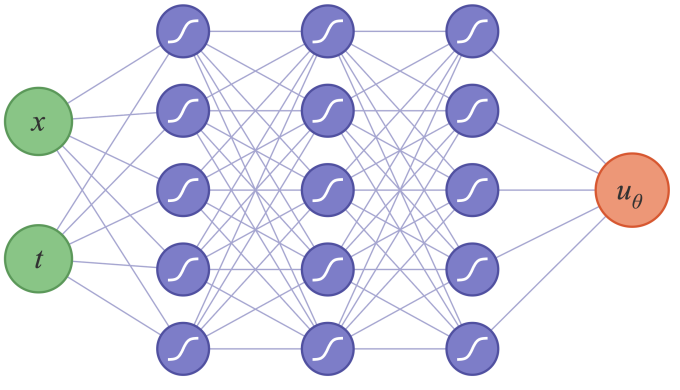
Physics-Informed Neural Network

Implicit Neural Representation (or Coordinate-based Network)

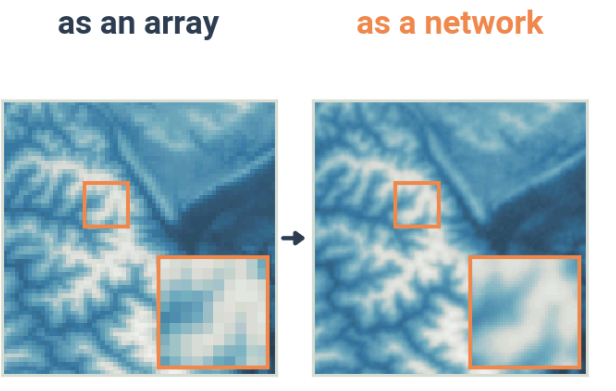
- So far, a NN was a **predictor**: encode *rules* extracted from a dataset to map a sample to a label
- INR: the network maps **coordinates** to the **value of a field** at those coordinates

$$h_\theta : (\mathbf{x}, t) \longmapsto u(\mathbf{x}, t)$$

- The NN no longer encode rules learned from many examples, it encodes **one single object**.



The network takes coordinates as input and outputs the value of the field at those coordinates



64 × 64 values

only at the nodes

f_θ : 131 000 weights

everywhere, any resolution

Notebook lec6-inr.ipynb

Implicit Neural Representation (or Coordinate-based Network)

Three consequences, and they are the whole point:

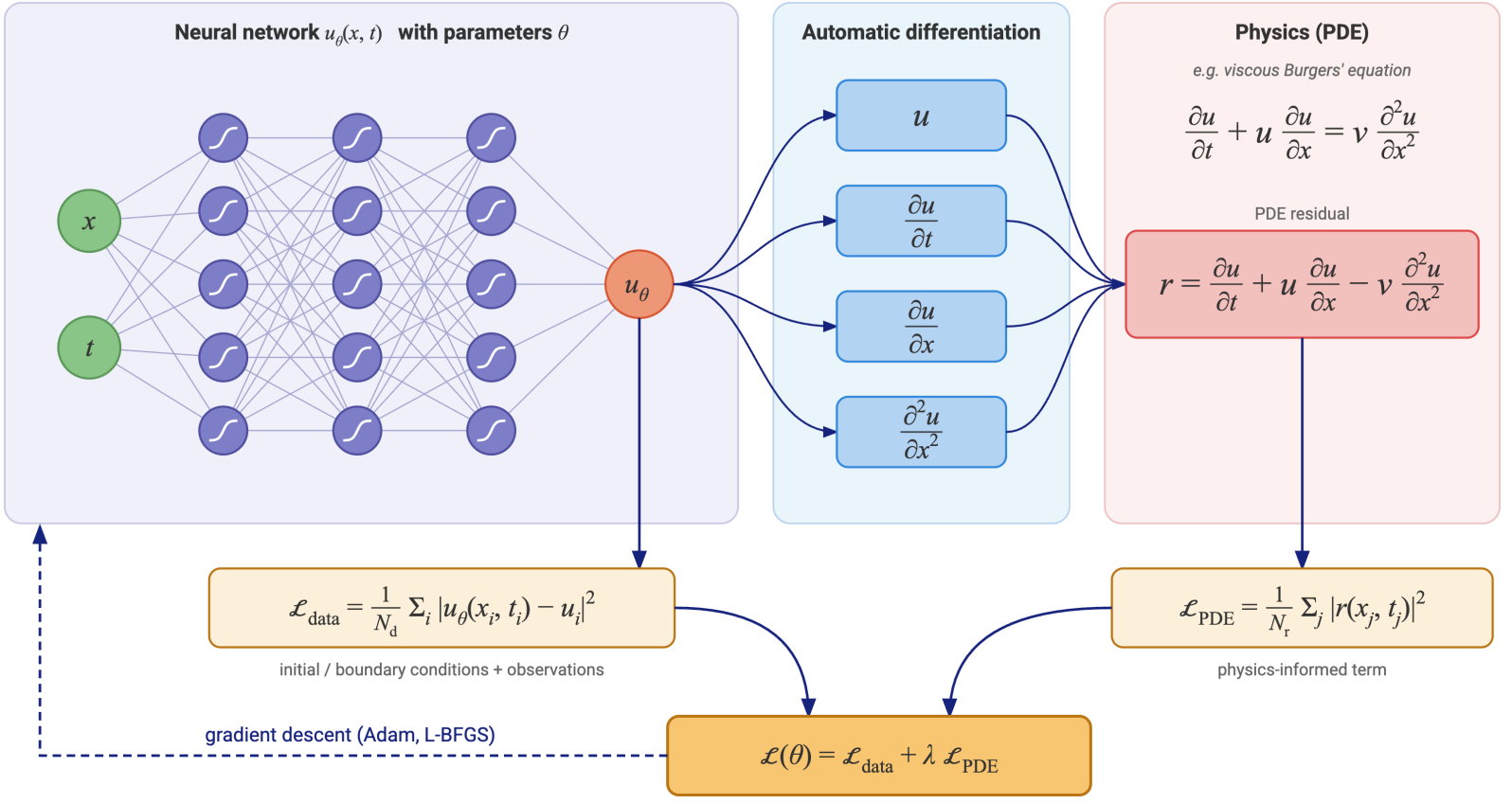
Continuous — the field is defined everywhere, not only at the nodes of a grid. Query any $\mathbf{x}$, at any resolution.

Compact — the memory cost is the number of weights, no longer the number of pixels or mesh cells.

Differentiable — $\partial h_\theta / \partial \mathbf{x}$ is available *exactly*, by automatic differentiation.

This last property is the one that changes everything.

Physics-Informed Neural Network (PINN) [1]



(1) Maziar Raissi, Paris Perdikaris, George Em Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations", Journal of Computational Physics, 2019

The residual loss

Given a PDE written as $\mathcal{N}[u] = 0$ on a domain Ω, with boundary and initial conditions:

$$\mathcal{L}(\theta) = \lambda_r \mathcal{L}_r(\theta) + \lambda_b \mathcal{L}_b(\theta) + \lambda_d \mathcal{L}_d(\theta)$$

PDE residual

$$\mathcal{L}_r = \frac{1}{N_r} \sum_{i=1}^{N_r} \left| \mathcal{N}[u_\theta(\mathbf{x}_i, t_i)] \right|^2$$

Boundary / initial

$$\mathcal{L}_b = \frac{1}{N_b} \sum_{j=1}^{N_b} \left| u_\theta(\mathbf{x}_j, t_j) - g_j \right|^2$$

Measurements

$$\mathcal{L}_d = \frac{1}{N_d} \sum_{k=1}^{N_d} \left| u_\theta(\mathbf{x}_k, t_k) - u_k \right|^2$$

- The $\mathbf{x}_i$ are **collocation points**, sampled anywhere in Ω — no mesh, and **no labels needed**
- The third term is **optional**: a PDE can be solved with zero measurements

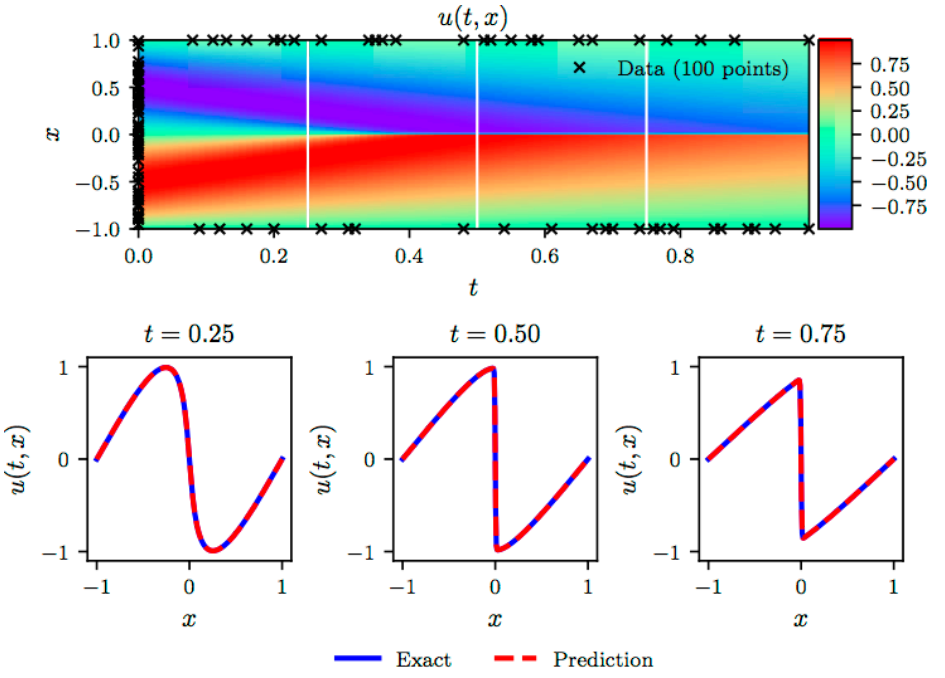
Example: the Burgers' equation

Given $\nu = 0.01/\pi$, $x \in [-1, 1]$, $t \in [0, 1]$ and $u : [-1, 1] \times [0, 1] \to \mathbb{R}$:

$$\begin{cases} \frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} - \nu\frac{\partial^2 u}{\partial x^2} = 0 \\ u(x, 0) = -\sin(\pi x) \\ u(-1, t) = u(1, t) = 0 \end{cases}$$

Right: INR trained with

- 100 points at $t = 0$, $x = -1$ and $x = 1$
- $10,000$ collocation points sampled in the domain $[-1, 1] \times [0, 1]$



PINN in practice

Several practical issues arise when training a PINN:

- **Hard to train**
  The "data" and "physics" terms are difficult to balance during training, leading to convergence issues
- **Poor performance for large temporal and spatial scales**
  Usually fail to capture long-term temporal dynamics and large spatial domain needs very deep networks and a huge amount of collocation points
- **High computational cost**
  Training PINNs can be computationally expensive, especially for large-scale problems
- **Fail to capture multi-scale dynamics**
  Many spatio-temporal phenomena exhibit behavior across multiple scales, which is challenging for PINNs to capture accurately
- **Limited generalization ability**
  Models may not generalize well to unseen regions or different parameter values

Notebook lec6-pinn-oscillator.ipynb

What PINNs are good for — and where they stop

Inverse problems — an unknown physical parameter μ becomes a *trainable variable*, optimized jointly with θ. No outer loop of forward solves.

Sparse and noisy measurements — the PDE acts as a physical regularizer and fills the gaps: a full velocity field from a handful of sensors.

Awkward geometries — nothing to mesh, and the cost does not explode with the dimension of the domain.

But one network solves one problem instance. Change the initial condition, the boundary conditions or a coefficient, and everything has to be retrained from scratch.

A PINN is a **solution**. What we would really like is a **solver**.

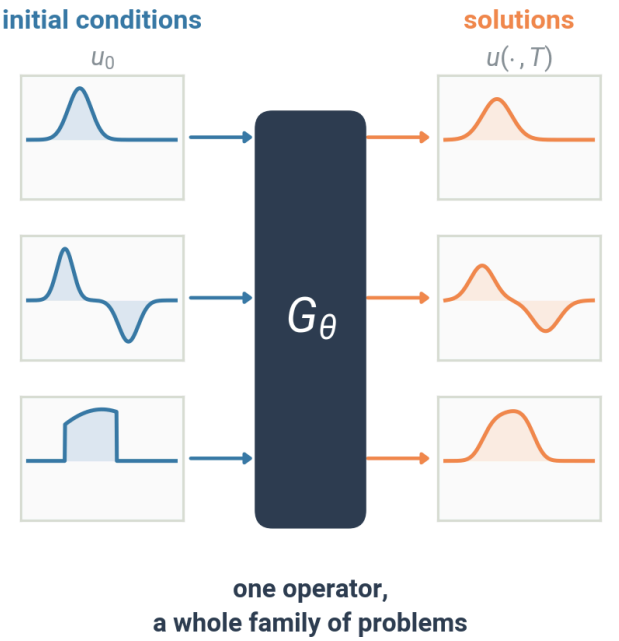
Neural Operators

Learning the solution operator

- PINN = **one** triple (μ, g, u_0).
- Change the initial condition and it needs to be retrained
- Solver = **map between functions**

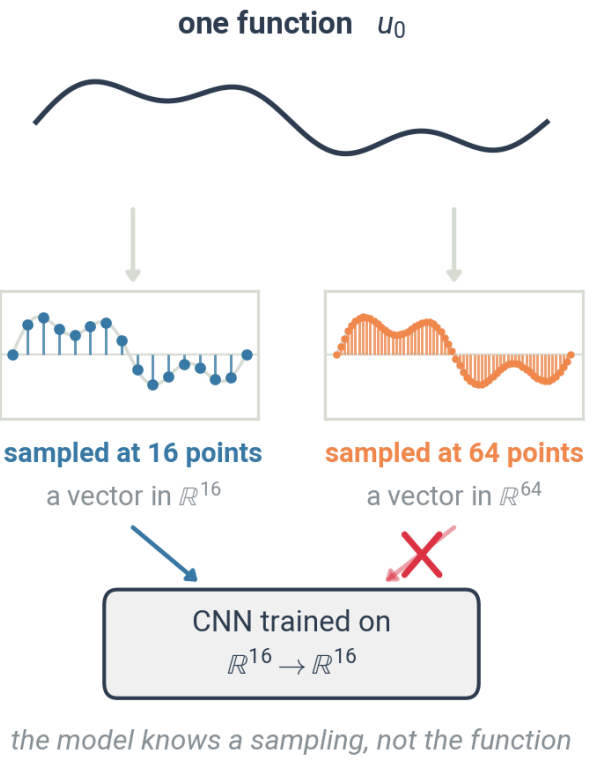
$$\mathcal{G} : u_0 \longmapsto u$$

  from the initial condition to the whole solution
- A **neural operator** learns $\mathcal{G}$ itself, once, from a set of instances already solved by a classical solver



Why a convolutional network is not enough

- Natural idea: sample u_0 on a 64×64 grid, feed it to a CNN, output u on the same grid. **It works.**
- But what has been learned is a map between **vectors of fixed size**, not between functions
- Change the resolution, the mesh or the geometry, and the model is useless



the model knows a sampling, not the function

What we want is **discretization invariance**: a model trained at one resolution that can be evaluated at another, because it has learned something about the *functions*, not about the *samples*.

Learning in function space

- A neural operator maps a **function** to a **function**

$$\mathcal{G} : a \in \mathcal{A} \longrightarrow u \in \mathcal{U}$$

  where $\mathcal{A}$ and $\mathcal{U}$ are **infinite-dimensional** spaces of functions over the domain.
- NN layers have finite numbers of neurons, and the update rule is a sum over neurons:

$$v_{t+1,i} = \sigma\Big(\sum_{j=1}^{n} W_{ij}\, v_{t,j} + b_i\Big)$$

- We need layers that operates on **functions**, not on **vectors**. The sum becomes an integral over the domain:

$$v_{t+1}(\mathbf{x}) = \sigma\Big(\mathbf{W} v_t(\mathbf{x}) + \int_{\Omega} \kappa(\mathbf{x}, \mathbf{y})\, v_t(\mathbf{y})\, d\mathbf{y}\Big)$$

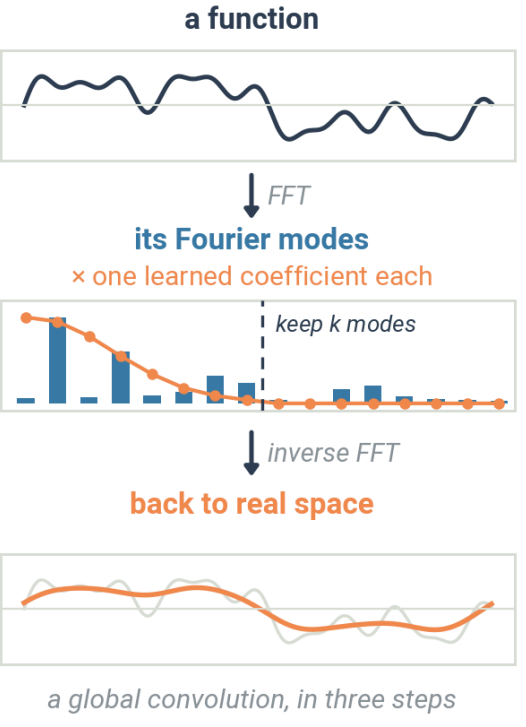
- Nothing in this formula mentions a grid: the architecture is defined on the *functions*, and any discretization is just a way to evaluate the integral
- The challenge: **how do we compute that integral cheaply?**

Fourier Neural Operator: the idea [1]

- For a linear PDE, the solution is a convolution with the Green's function

$$u(\mathbf{x}) = \int G(\mathbf{x} - \mathbf{x}')\, f(\mathbf{x}')\, d\mathbf{x}'$$

- So the solution operator is, in essence, a **global convolution**
- And a global convolution is a **pointwise product in Fourier space**
- So: go to Fourier, multiply by learned coefficients, come back

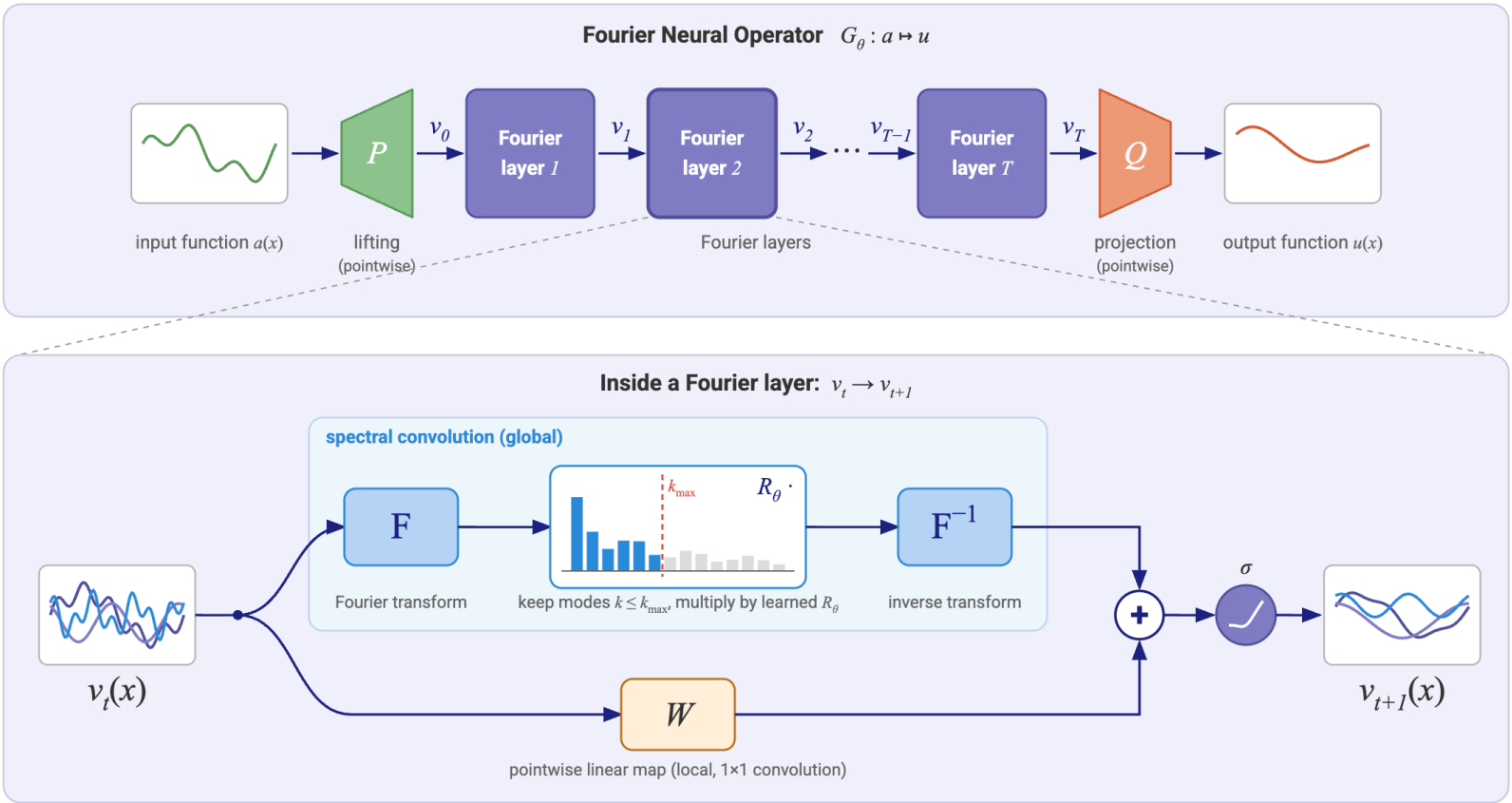


a global convolution, in three steps

Lecture 4: a CNN kernel is **local** (3×3) and defined on the grid. A Fourier kernel is **global** and defined on *frequencies* — a property of the function, not of the sampling.

(1) Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, Anima Anandkumar, "Fourier Neural Operator for Parametric Partial Differential Equations", ICLR 2021

Fourier Neural Operator: how it works



$$v_{t+1}(\mathbf{x}) = \sigma\Big(\underbrace{\mathbf{W} v_t(\mathbf{x})}_{\text{local}} + \underbrace{\mathcal{F}^{-1}\big(R_\phi \cdot \mathcal{F}(v_t)\big)(\mathbf{x})}_{\text{global}}\Big)$$

50 shades of Neural Operators

FNO is just one example of a neural operator where the integral is computed in Fourier space. Other approaches exist, e.g.:

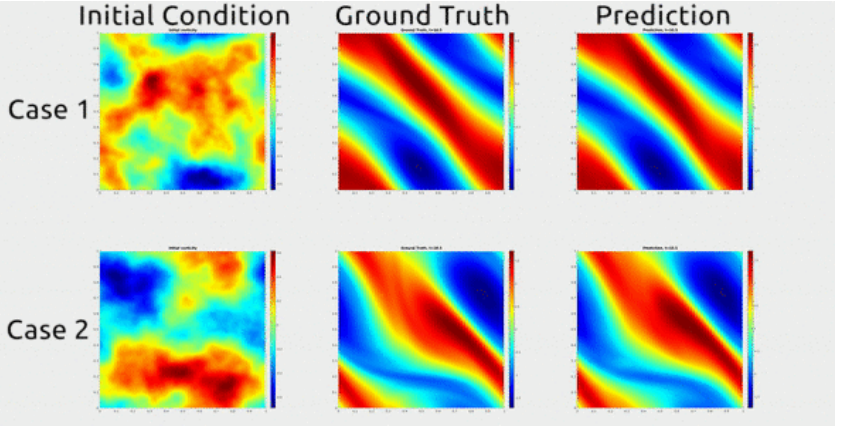
3.2.1 Deep Operator Network			
DeepONet [48]	*Nat. Mach. Intell.*	MLP	Reaction–Diffusion, other ODEs
PI-DeepONet [49]	*Sci. Advection*	MLP	Anti-Derivative, Reaction–Diffusion, Burgers, Eikonal, Transonic
MIONet [50]	*Siam J Sci Comput*	MLP	ODE, Anti-Derivative, Reaction–Diffusion
Fourier-MIONet [13]	*Arxiv 2023*	MLP	Gas saturation
NOMAD [52]	*NeurIPS 2022*	MLP	Advection, Shallow Water, Anti-Derivative
Shift-DeepONet [53]	*ICLR 2023*	MLP	Advection, Burgers, Euler
HyperDeepONet [54]	*ICLR 2023*	MLP	Advection, Burgers, Shallow Water
B-DeepONet [51]	*J. Comput. Phy.*	MLP	Anti-Derivative, Reaction–Diffusion, Advection
SVD-DeepONet [56]	*Comput Method Appl M*	MLP	ODEs
L-DeepONet [55]	*Arxiv 2023*	MLP	Rayleigh-Bénard, Shallow Water
3.2.2 In Physical Space			
MGNO [57]	*NeurIPS 2020*	GNN	Darcy, Burgers
Geneva *et al.* [58]	*Neural Networks*	Transformer	Navier-Stokes, Lorenz
GNOT [59]	*ICML 2023*	Transformer	Transonic, Elastodynamic, Navier-Stokes, Darcy
CNO [60]	*NeurIPS 2023*	CNN	Poisson, Allen-Cahn, Navier-Stokes, Darcy
FactFormer [158]	*NeurIPS 2023*	Transformer	Kolmogorov flow, Smoke buoyancy
LNO [61]	*Arxiv 2023*	Laplace	Diffusion, Duffing, Reaction–Diffusion
KNO [62]	*Arxiv 2023*	Koopman	Bateman–Burgers, Navier-Stokes
ICON [63]	*PNAS*	Transformer	Poisson, Reaction–Diffusion, ODEs
Transolver [159]	*ICML 2024*	Transformer	CarDesign, Airfoil
3.2.3 In Fourier Space			
FNO [64]	*ICLR 2021*	Fourier	Burgers, Darcy, Navier-Stokes
PINO [65]	*ACM/IMS Trans. Data Sci.*	Fourier	Burgers, Darcy, Navier-Stokes, Kolmogorov
GEO-FNO [66]	*JMLR*	Fourier	Advection, Elastodynamic, Airfoil, Navier-Stokes
U-NO [67]	*TMLR*	Fourier	Darcy, Navier-Stokes
SSNO [68]	*IEEE Access*	Fourier	Burgers, Darcy, Navier-Stokes
F-FNO [69]	*ICLR 2023*	Fourier	Kolmogorov, Transonic
CFNO [70]	*ICLR 2023*	Fourier	Navier-Stokes, Shallow Water, Maxwell
CMWNO [71]	*ICLR 2023*	Fourier	Gray-Scott
MCNP [72]	*Arxiv 2023*	Fourier	Diffusion, Allen-Cahn, Navier-Stokes
G-FNO [73]	*ICML 2023*	Fourier	Navier-Stokes, Shallow Water
GINO [74]	*NeurIPS 2023*	Fourier	Ahmed-Body, Reynold Averaged Navier-Stokes
DAFNO [75]	*NeurIPS 2023*	Fourier	Hyperelasticity, Airfoil
FCG-NO [76]	*ICML 2024*	Fourier	Poisson, Diffusion

Wang et al. ,"Recent Advances on Machine Learning for Computational Fluid Dynamics: A Survey", 2025

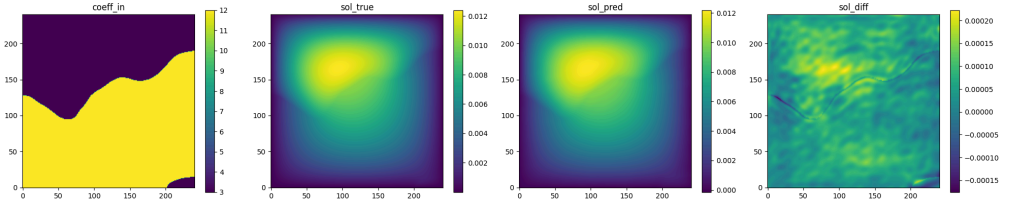
50 shades of Neural Operators

NO can be used for different types of tasks since $a \in \mathcal{A}, u \in \mathcal{U}$ can be different types of functions, e.g.:

- An initial condition and a solution for a time dependent PDE



- A parameter function and a solution for a time independent PDE



- A solution at a coarse resolution and a solution at a fine resolution (super-resolution)

Neural Operators vs traditional solvers

Where NOs win:

- **Low-to-moderate accuracy requirements**: outperforms solvers with massive speedups of 10^3 to 10^5 times faster.
- **Many-query scenarios** : engineering tasks requiring thousands of simulations, such as structural optimization, climate forecasting, or real-time digital twins
- **Resolution Invariance (Zero-Shot Super-Resolution)**: a model trained on a coarse grid can be evaluated on a finer grid without retraining, which is not possible with traditional solvers.

Where solvers win:

- **High-Precision/Safety Critical**: solvers can achieve machine precision, while NOs are limited to a few digits of accuracy.
- **Error Accumulation**: solvers can control and minimize error accumulation over time, while NOs may suffer from error propagation.
- **Out-of-Distribution Failures**: solvers are robust to changes in problem parameters, while NOs may fail when tested on scenarios outside their training distribution.

What else in ML for Physics?

The other two ways to incorporate scientific constraints

1. **Soft constraint** : The equation in the loss, this is the PINN we have just seen.

2. Example of **Hard constraint** :

   - **Hamiltonian Neural Networks** (Greydanus et al., 2019) and **Lagrangian Neural Networks** (Cranmer et al., 2020) learn the *Hamiltonian* (potential energy plus kinetic energy) or *Lagrangian* (kinetic energy minus potential energy) instead of the dynamics itself, and then use classical mechanics to compute the dynamics → the network is guaranteed to conserve energy

3. Example of **Hybrid** :

   - **Neural Ordinary Differential Equations** (Chen et al., 2018): $\frac{d\mathbf{z}}{dt} = NN(\mathbf{z}, t, \theta)$ is a neural network and the solution is computed by a classical ODE solver
   - **Hamiltonian Neural Networks** and **Lagrangian Neural Networks** also belong to this category
   - **ML-accelerated solvers**, e.g. with learned preconditionners (Lan et al. 2024)

What else in ML for Physics?

The other two scientific tasks

Recall the three tasks: finding u, finding μ, finding $\mathcal{N}$. We only developed the first one.

Inverse problems — finding μ

- **Trainable-parameter trick**: the unknown parameter μ is a trainable variable, optimized jointly with the network weights θ — PINN, learned preconditioner
- **Neural operator**: the unknown parameter μ is a function, and the neural operator learns the map from μ to u — FNO, DeepONet

Discovery — finding $\mathcal{N}$

- **Symbolic regression**: the unknown operator $\mathcal{N}$ is a symbolic expression, and the network learns to predict the coefficients of a library of candidate terms — PDE-FIND, SINDy
- **Neural operator**: the unknown operator $\mathcal{N}$ is a function, and the neural operator learns the map from u to $\mathcal{N}[u]$ — DeepONet, FNO

But there exists many other ways to solve inverse problems and discover new laws, e.g. by combining neural networks with classical solvers, or by using reinforcement learning.

What else in ML for Physics?

And the questions that are still open

Uncertainty quantification — everything we have built returns a single number. A prediction without an error bar is not a physical result.

Generalization — these models interpolate inside their training envelope. Extrapolating to a new regime is exactly what physics asks for, and what they do worst.

Geometry — the FFT of an FNO needs a regular grid. Real engineering lives on unstructured meshes, which is where graph-based operators come in.

Data — a neural operator needs a classical solver to produce its training set. Who pays for it, and on which benchmark do we compare?

Takeaways

Scientific machine learning is not generic machine learning. The goal is not only to score well on a test set, but to stay consistent with known laws and to hold up in regimes never observed.

Scientific knowledge can be injected **in the loss**, **in the architecture**, or **in the solver**. This lecture developed the first route; the other two are just as active.

A PINN is a solution. The network *is* the field, training *is* solving, and the residual needs no labels at all — only derivatives with respect to the inputs. But one network solves one problem instance.

A neural operator is a solver. It maps a function to a function: the sum over neurons becomes an integral over the domain, and that is exactly what frees it from any grid.

Neither replaces the classical solver. On a plain forward problem a solver is still faster and more accurate, and a neural operator needs one to generate its training data. They win where the classical route is structurally expensive: inverse problems, sparse measurements, whole families of instances.